

TAKE-HOME CODING CHALLENGE

Full-Stack CRUD Application

TypeScript · React · Fastify · PostgreSQL

Time allowed	Submission	Format	Level
72 hours	GitHub / ZIP	Monorepo	Mid – Senior

1. Welcome

Thank you for taking the time to complete this challenge. It is designed to reflect the kind of work you would do day-to-day on our team — not puzzle-solving or whiteboard tricks. We want to see how you design, structure, and ship a real feature.

What we are looking for

We are not looking for perfection. We are looking for clean, readable, type-safe code with honest trade-offs. A smaller scope done well beats a large scope done carelessly.

You will build a small but complete application using the same stack we run in production:

- TypeScript 5+ (strict mode, no any allowed)
- React 18+ with React Query for server state
- Fastify 4+ with Zod for validation and typed routes
- PostgreSQL 15+ with Prisma or Drizzle ORM

Time guidance

This challenge is designed to take 6–8 hours of focused work. You have 72 hours from receipt so you can fit it around your schedule. Do not spend more than 10 hours — if you are still working at that point, submit what you have with notes on what you would finish next.

2. The Brief

You are building a Candidate Tracker — a lightweight internal tool for managing job applicants and their individual interview applications. The product has two entities: Candidates and Applications. A Candidate is a person; an Application is one job they applied for. One candidate can have multiple applications.



Entity relationship

Candidate (parent) → Applications (children). Each Application belongs to exactly one Candidate. When viewing an Application you must be able to navigate to the parent Candidate's detail view.

2.1 What you are building

The application has four main areas:

Area	Description
Dashboard	An overview page with key metrics and recent activity. The first thing users see when they open the app.
Candidates	Full CRUD for the Candidate entity. Search by candidate fields. Link through to their Applications.
Applications	Full CRUD for the Application entity. Search by application fields and by candidate fields (name, email). Clicking an application shows the parent candidate.
Candidate detail	A single candidate page that shows the candidate's information and a list of all their applications.

2.2 Entity fields

Design your own schema and migrations. The following fields are required on each entity — you are free to add more if it makes the application feel more complete.

Candidate fields

Field	Type	Notes
id	UUID	Primary key, generated by the database
name	Text	Required. Full name of the person
email	Text	Required. Unique across all candidates
phone	Text	Optional
location	Text	Optional. City or country
linkedin_url	Text	Optional. URL to their LinkedIn profile

notes	Text	Optional. Free-form internal notes
created_at	Timestamp	Set by the database on insert
updated_at	Timestamp	Updated automatically on every change

Application fields

Field	Type	Notes
id	UUID	Primary key, generated by the database
candidate_id	UUID	Foreign key to Candidates. Required
job_title	Text	Required. Title of the role they applied for
company	Text	Required. Company name
status	Enum	applied → screening → interview → offer → hired / rejected
applied_at	Date	Required. The date they submitted the application
salary_expectation	Integer	Optional. In the candidate's local currency
source	Text	Optional. How they found the role (LinkedIn, referral, etc.)
notes	Text	Optional. Free-form notes about this specific application
created_at	Timestamp	Set by the database on insert
updated_at	Timestamp	Updated automatically on every change

🔧 You may extend these

If you want to add extra fields that make the application feel more complete, go ahead. Document your additions in the README and explain why you added them.

3. Dashboard

The dashboard is the home page of the application. It must give the user a quick, at-a-glance picture of the current state of the pipeline without them needing to open any other page.

3.1 Required metrics

The dashboard must display the following metrics:

- Total candidates
- Total applications
- Applications by status
- Hired this month
- Rejection rate
- Latest applications

3.2 Chart

Include at least one chart on the dashboard. Suggested options:

- A bar chart of application counts grouped by status
- A line chart of applications created per week over the last 8 weeks
- A doughnut chart showing the status distribution

Use any charting library you are comfortable with (Recharts, Chart.js, Visx, Victory). The chart must be responsive and readable on a standard laptop screen.

Dashboard tip

All dashboard numbers must be fetched from the API — not calculated in the frontend from a full data dump. Design your endpoints to return aggregated data efficiently from the database.

4. CRUD Requirements

Both entities require full create, read, update, and delete support. The requirements below apply to both Candidates and Applications unless stated otherwise.

4.1 Candidates

Op	Detail
Create	Form with all candidate fields. Validates email uniqueness before saving.
List	Paginated table. Search by name, email, location, or phone. Each row links to the candidate detail page.
Detail	Full candidate profile page showing all fields and a table of their applications. Each application row links to the application detail or edit view.
Update	Edit form pre-filled with current values. All fields editable except id and created_at.
Delete	Soft delete (set deleted_at). Requires a confirmation step. Soft-deleted candidates must not appear in any list or search result.

4.2 Applications

Op	Detail
Create	Form requires selecting an existing candidate (dropdown or search-as-you-type). All application fields present.
List	Paginated table of all applications. See section 4.3 for search and filter requirements.
Detail / Edit	Shows all application fields. Must display the parent candidate's name and email as a clickable link to their detail page.
Update	Edit form for all application fields. Candidate assignment (candidate_id) may also be changed.
Delete	Hard or soft delete acceptable. Requires a confirmation step.

4.3 Cross-entity search on the Applications list

This is one of the key technical requirements. The Applications list page must allow users to search across both the Application's own fields and the parent Candidate's fields in a single search box.

What the search must cover

A user typing 'Alice' into the search box should find applications where the job title matches, the company matches, OR the linked candidate's name or email matches — all from one query. The join must happen on the server, not by fetching all data and filtering in JavaScript.

Searchable fields on Applications list

- Application: job_title, company, source, notes
- Parent Candidate: name, email, location

Filterable fields on Applications list

- Status (single select from the enum values)
- Applied date range (from / to)

All search and filter parameters must be sent as query parameters to the backend. The backend must perform the search with a JOIN and return only matching results. Implement debouncing on the search input in the frontend.

5. Frontend Requirements

The React application must be functional and usable. We are not assessing your visual design skills, but the UI must be clear, handle all states correctly, and feel professional. Use Tailwind CSS or any component library you are comfortable with.

5.1 State & data fetching

Use React Query (TanStack Query) for all server state. You must demonstrate:

- `useQuery` for fetching lists with active filters and search terms
- `useMutation` for create, update, and delete operations
- Cache invalidation after mutations so lists refresh automatically
- Dashboard metrics fetched from dedicated aggregation endpoints — not derived from full list queries

5.2 UI states — all must be handled

State	Required behaviour
Loading	Skeleton or spinner shown while data is fetching. No blank white screen.
Empty	Friendly message when no records match the current search or filters.
Error	User-visible error message when the API call fails. Offer a retry action.
Submitting	Form submit button shows a loading state and is disabled to prevent double-submit.
Delete confirm	A confirmation prompt before deleting. Do not delete on first click.

5.3 Navigation

- Dashboard → links to both the Candidates list and Applications list
- Candidates list → each row links to the Candidate detail page
- Candidate detail → each application in the list links to the Application edit/detail view
- Applications list → each row shows the parent candidate's name as a clickable link to their detail page
- Application edit/detail → a visible link back to the parent Candidate's detail page

6. Technical Requirements

These are non-negotiable. Every item below is something we apply in our production codebase and will look for in your submission.

6.1 TypeScript

- strict: true in tsconfig.json on both frontend and backend
- No any. Use unknown with type narrowing, or Zod inference
- All Fastify route request and response types must be inferred from Zod schemas
- React component props must have explicit types
- Shared types (Candidate, Application) must live in a shared package imported by both apps

6.2 Backend

- Use Fastify's schema-first approach with fastify-type-provider-zod
- Use Prisma or Drizzle for all database access — no raw SQL except in migrations
- Search must use parameterised queries, never string concatenation
- Cross-entity search must use a JOIN in SQL, not multiple queries merged in JavaScript
- All errors must go through a single Fastify setErrorHandler — no ad-hoc try/catch in routes
- Enable Fastify's built-in request logging

6.3 Database

- Schema changes must be done via migration files (Prisma migrate or Drizzle migrate)
- Add appropriate indexes on columns used for search and filtering
- Soft delete on Candidates: use a deleted_at column, filter it in all queries
- The seed script must populate at least 10 candidates, each with 2–4 applications

6.4 Project structure

We expect a monorepo with the following layout:

```
candidate-tracker/
├── apps/
│   ├── api/          # Fastify backend
│   └── web/          # React frontend
├── packages/
│   └── shared/        # Shared TypeScript types & Zod schemas
├── package.json      # Root workspace config
└── README.md
```


 Shared types tip

Define your Zod schemas once in packages/shared. Both the Fastify backend and the React frontend import from there. The backend infers its request/response types; the frontend infers its form and API response types. This is the single-source-of-truth pattern we use in production.

7. Testing

We require tests. Not 100% coverage — just meaningful tests that give us confidence the core behaviour works and that show you know how to write them.

7.1 Required tests

What to test	How
POST /api/candidates	201 on valid body, 400 on invalid body, 409 on duplicate email.
GET /api/candidates	Returns paginated list. Search by name filters results correctly.
GET /api/applications (search)	Searching by candidate name returns applications linked to matching candidates.
POST /api/applications	201 with valid candidate_id, 400 with missing required fields, 404 with unknown candidate_id.
PATCH /api/applications/:id	200 on valid update, 404 on unknown ID.
GET /api/dashboard	Returns all required metric fields with correct types.
Zod schemas	Unit test the shared schemas: valid input passes, each invalid case fails with the correct error message.

7.2 Recommended tools

- Backend tests: Vitest or Jest with Fastify's inject() — no running server needed
- Database: use a test database or SQLite for tests — not the dev database
- Frontend component tests with React Testing Library are a bonus

☒ Test tip

Four solid route tests are worth more than thirty shallow snapshot tests. Focus your testing effort on the cross-entity search endpoint and the dashboard aggregation endpoint — those are the most complex and the most likely to break.

8. Setup & Running Locally

We will clone your repository and run it. Make sure it works from a clean checkout with the commands below. If your setup requires anything unusual, document it clearly in the README.

8.1 Expected commands

```
# 1. Install all dependencies
npm install

# 2. Start the database
docker compose up -d

# 3. Run migrations and seed data
npm run db:migrate
npm run db:seed

# 4. Start the API (port 3001)
npm run dev --workspace=apps/api

# 5. Start the web app (port 5173)
npm run dev --workspace=apps/web

# 6. Run all tests
npm test
```

8.2 Docker Compose

Include a docker-compose.yml for PostgreSQL so reviewers do not need a local Postgres installation:

```
services:
  postgres:
    image: postgres:15-alpine
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: candidate_tracker
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

8.3 Environment variables

Include a `.env.example` file. Never commit real credentials.

```
# apps/api/.env.example
DATABASE_URL=postgresql://dev:dev@localhost:5432/candidate_tracker
PORT=3001
NODE_ENV=development

# apps/web/.env.example
VITE_API_URL=http://localhost:3001
```

9. How We Score Your Submission

We review every submission on the same rubric. Here is exactly what we look at and how much weight each area carries.

Area	Weight	What reviewers look for
TypeScript quality	15%	No any, strict mode, inferred types end-to-end, shared schemas between front and back.
Backend correctness	20%	All CRUD endpoints work. Validation enforced. Correct HTTP status codes. No SQL injection.
Cross-entity search	15%	Applications search queries both application and candidate fields via a server-side JOIN. Not filtered in JS.
Dashboard	15%	All 6 metrics present. Chart included. Data aggregated on the server. Updates when data changes.
Database design	10%	Migrations present. Seed data. Appropriate indexes. Soft delete implemented correctly.
Frontend UX	15%	All CRUD works. All UI states handled. Navigation between entities works. Search debounced.
Tests	10%	Required route tests pass. Cross-entity search tested. Schemas unit-tested.

9.1 Bonus points (not required)

- Optimistic updates on status change or delete
- Cursor-based pagination instead of offset-based
- A Kanban board view of applications grouped by status
- Frontend component tests with React Testing Library
- A Dockerfile for the API service

What impresses us most

Candidates who score highest are not those who add the most features. They are the ones whose code is easy to read, whose error handling is thorough, whose types are tight, and whose README explains their decisions honestly — including what they did not finish and why.

10. Submitting Your Work

10.1 Checklist before you submit

☐	npm install && npm test runs without errors from the repo root
☐	docker compose up -d starts PostgreSQL successfully
☐	npm run db:migrate && npm run db:seed runs without errors
☐	The API starts on port 3001 and CRUD endpoints for both entities respond correctly
☐	The web app starts on port 5173 — dashboard, candidates, and applications all load
☐	Cross-entity search on the Applications list returns results matching candidate fields
☐	Clicking a candidate name in the Applications list navigates to the Candidate detail page
☐	TypeScript compiles with no errors: npx tsc --noEmit in both apps
☐	.env.example files are present and documented in the README
☐	No secrets, passwords, or real .env files committed to the repository
☐	README contains setup steps, architectural decisions, and what you would do with more time

10.2 How to submit

1. Push your code to a public or private GitHub repository.
2. If private, invite our GitHub handle (provided separately) as a collaborator.
3. Reply to the email you received this document in with the repository URL.
4. Include in the email body: estimated time spent and any notes you want us to read first.

What to include in your README

Your README is part of the submission. Cover: (1) how to run the project, (2) non-obvious architectural decisions, (3) what you would improve or finish given more time, (4) anything that does not work and why. Honest README notes are valued, not penalised.

11. Questions

If something in this brief is unclear, email us before you start coding rather than making an assumption that leads you in the wrong direction. If you choose to resolve an ambiguity yourself, document your interpretation in the README.

A final note

We know take-home challenges take time, and we respect that. In return we commit to: reviewing your code thoroughly, giving you personalised written feedback regardless of the outcome, and scheduling a 30-minute debrief call where you can ask us questions about the review. Good luck — we look forward to reading your code.